

How the Market-Research PDFs Were Generated

Wherabouts.com

June 5, 2026

How the Market-Research PDFs Were Generated

A note on the tooling behind `COMPREHENSIVE-REPORT.pdf`, `EXECUTIVE-BRIEF.pdf`, and `PERSONAS.pdf`

The PDFs were not produced by a single “PDF library.” They came out of a small pipeline run by the `make-pdf` tool — a Bun-compiled TypeScript CLI. Here is exactly what does the work, stage by stage.

The pipeline

1. `marked` — the one real third-party JavaScript dependency in the renderer. It parses the markdown source into HTML.
2. **Custom sanitizer + `smartypants.ts`** (in-house) — strips unsafe tags (`<script>`, `<iframe>`, `javascript:` URLs, and similar) and converts straight quotes and dashes into curly quotes and em dashes, so copy-paste from the PDF produces clean typography.
3. **Custom `print-css.ts`** (in-house) — the print stylesheet that produces the “publication look”: 1-inch margins, running headers, page numbers, the cover page, and the table-of-contents styling. This is hand-written CSS, not a library theme.
4. **Headless Chromium** — the actual HTML-to-PDF rendering engine. The CLI shells out to gstack’s own `browse` binary (via `browseClient.ts`), which drives Chromium’s print-to-PDF over the DevTools protocol. This is the step that rasterized the three files.
5. `pdftotext` (**Poppler**) — *not* used during generation. It is only a CI/test gate that re-extracts text from a finished PDF to verify that copy-paste yields clean words (e.g. `Sailing`) rather than fragmented output (e.g. `S a i l i n g`).

The short version

`marked` for parsing → custom CSS for layout → headless Chromium for rendering.

- No Puppeteer or Playwright npm package — the tool talks to Chromium directly through the `browse` daemon.
- No LaTeX.

- The skill’s own documentation mentions “Paged.js” in the context of a pagination timeout error, but it is not imported in the current renderer source. This build relies on Chromium’s native print pagination plus the hand-written print CSS.

Runtime

The `make-pdf` binary itself is compiled with **Bun**, which is also why binary lookups in the source use `Bun.which(...)` for locating the `browse` and `pdftotext` executables across platforms.